

1. Layout-Grundlagen

Layout = Struktur und Anordnung von Elementen auf einer Webseite (wichtig für Lesbarkeit) Drei Hauptmethoden: Float (historisch), Flexbox (1D), Grid (2D)

2. Floats

Zweck: Text um Bilder fließen lassen | Element verlässt Dokumentenfluss

Problem: Container kollabiert → Lösung: overflow: hidden; oder Clearfix mit ::after {



```

1  a {
2    float: left | right | none;
3    clear: both;      /* Verhindert Elemente daneben */
4  }

```

clear: both; } Grenzen: Keine vertikale Zentrierung, unberechenbar → Flexbox/Grid bevorzugen

3. Flexbox (Zeile ODER Spalte)



```

1  .container {
2    display: flex;
3    flex-direction: row | column | row-reverse;
4    justify-content: flex-start | center | space-between | space-around | space-evenly;
5    align-items: stretch | center | flex-start | flex-end;
6    flex-wrap: wrap;
7    gap: 20px;
8  }
9  .item { flex: 1; } /* Größenverhältnis: flex: 1 vs flex: 3 */

```

Hauptachse (Main-Axis): **justify-content** | Querachse (Cross-Axis): **align-items**

4. CSS Grid (Zeilen UND Spalten)



```

1  .grid {
2    display: grid;
3    grid-template-columns: 200px 1fr 1fr; /* 3 Spalten */
4    grid-template-rows: 100px auto;
5    gap: 20px;
6  }

```

Methode 1 - Grid-Linien:



```

1  .header { grid-column: 1 / 4; grid-row: 1 / 2; }

```

Methode 2 - Benannte Bereiche:



```

1  .container {
2    grid-template-areas:
3      "header header header"
4      "nav   main   sidebar"
5      "footer footer footer";
6  }
7  .header { grid-area: header; }
8

```

5. Positionierung

```

1  a {
2    position: static;    /* Standard, normaler Fluss */
3    position: relative; /* Vershoben vom Original-Platz */
4    position: absolute; /* Relativ zum nächsten positionierten Eltern, verlässt Fluss */
5    position: fixed;    /* Relativ zum Viewport, scrollt nicht mit, verlässt Fluss */
6    position: sticky;   /* Wechselt zwischen relative und fixed */
7  }

```

Zentrierung:

px = absolut (fest) | % = relativ zum Elternelement

```

1  .centered {
2    position: absolute;
3    top: 50%;
4    left: 50%;
5    transform:
6      translate(-50%, -50%);
7  }
8

```

6. Farben

```

1  a {
2    color: red;           /* Name */
3    color: #ff0000;      /* HEX */
4    color: #f00;         /* HEX kurz */
5    color: rgb(255, 0, 0); /* RGB */
6    color: rgba(255, 0, 0, 0.5); /* RGBA - Alpha: 0.0 (transparent) bis 1.0 (deckend) */
7  }

```

color: rgba(255, 0, 0, 0.5); /* RGBA - Alpha: 0.0 (transparent) bis 1.0 (deckend) */

7. BEM (Block - Element - Modifier)

Warum? Klare Namen, keine Konflikte, wartbar, wiederverwendbar

```

1  .card { a }           /* Block: Eigenständige Komponente */
2  .card__title { a }    /* Element: Teil eines Blocks (mit __) */
3  .card__image { a }
4  .card--featured { a } /* Modifier: Variante/Zustand (mit --) */
5  .card__title--large { a }

```

Regeln: Kleinbuchstaben, keine Umlaute, __ für Elemente, -- für Modifier

8. Best Practices

☒ Semantisches HTML (<header>, <nav>, <main>, <footer>)
 ☒ BEM-Klassennamen für große Projekte
 ☒ Flexbox für 1D, Grid für 2D Layouts
 ☒ Kommentare + saubere Einrückung
 ☒ Ausreichender Farbkontrast
 ☒ Valider Code